

data_setup2.py

```
1 import os
2 import json
3 import numpy as np
4 import pandas as pd
5 from PIL import Image
6 from sklearn.model_selection import train_test_split
7
8 from config2 import (
9     seed_everything, SEED, TRAIN_DIR, OUTPUT_DIR, NORM_STATS_PATH,
10     CLASS_NAMES, CLASS_TO_IDX, VAL_SPLIT, IMAGE_SIZE,
11 )
12
13 IMG_EXT = (".png", ".jpg", ".jpeg", ".bmp")
14
15
16 def compute_train_stats(train_df: pd.DataFrame) -> dict:
17     """Mean/std of grayscale pixels over the TRAIN split only (in [0,1])."""
18     pixel_sum = 0.0
19     pixel_sq_sum = 0.0
20     n_pixels = 0
21     for fp in train_df["filepath"]:
22         img = Image.open(fp).convert("L").resize((IMAGE_SIZE, IMAGE_SIZE))
23         arr = np.asarray(img, dtype=np.float64) / 255.0
24         pixel_sum += arr.sum()
25         pixel_sq_sum += (arr ** 2).sum()
26         n_pixels += arr.size
27     mean = pixel_sum / n_pixels
28     var = pixel_sq_sum / n_pixels - mean ** 2
29     std = float(np.sqrt(max(var, 1e-12)))
30     return {"mean": float(mean), "std": std}
31
32
33 def main():
34     seed_everything(SEED)
35
36     print("=" * 65)
37     print(" DATA SETUP - scanning folders & stratified train/val split")
38     print("=" * 65)
39
40     # — 1. Collect every image path + label —————
41     records = []
42     for class_name in CLASS_NAMES:
43         class_dir = TRAIN_DIR / class_name
44         if not class_dir.exists():
45             print(f" [WARNING] Folder not found: {class_dir}")
46             continue
47         files = sorted(f for f in os.listdir(class_dir)
48                        if f.lower().endswith(IMG_EXT))
49         for fname in files:
50             records.append({
51                 "filepath": str(class_dir / fname),
52                 "label": CLASS_TO_IDX[class_name],
53                 "class_name": class_name,
54                 "filename": fname,
55             })
56
57     df = pd.DataFrame(records)
58     if len(df) == 0:
59         raise RuntimeError("No images found. Check TRAIN_DIR and folder names "
60                            "(they contain spaces: 'class 1'..'class 4').")
61
62     # — 2. Class distribution —————
63     print(f"\n Total images found : {len(df)}")
64     print(" Class distribution :")
65     for cls in CLASS_NAMES:
66         c = len(df[df["class_name"] == cls])
67         print(f" {cls} : {c:>4d} images ({c/len(df)*100:.1f} %)")
68
69     # — 3. Stratified split (fixed seed) —————
70     train_df, val_df = train_test_split(
```

```

71     df, test_size=VAL_SPLIT, random_state=SEED, stratify=df["label1"],
72 )
73 train_df = train_df.reset_index(drop=True)
74 val_df = val_df.reset_index(drop=True)
75
76 train_df.to_csv(OUTPUT_DIR / "train_split.csv", index=False)
77 val_df.to_csv(OUTPUT_DIR / "val_split.csv", index=False)
78 print(f"\n Train set : {len(train_df):>4d} images")
79 print(f" Val set : {len(val_df):>4d} images")
80
81 print("\n Stratification check (val) :")
82 for cls in CLASS_NAMES:
83     tot = len(df[df["class_name"] == cls])
84     v = len(val_df[val_df["class_name"] == cls])
85     print(f" {cls} : {v}/{tot} ({v/tot*100:.1f} % in val)")
86
87 # — 4. Train-only normalization stats (no leakage) —————
88 print("\n Computing normalization stats on TRAIN split only ...")
89 stats = compute_train_stats(train_df)
90 with open(NORM_STATS_PATH, "w", encoding="utf-8") as f:
91     json.dump(stats, f, indent=2)
92 print(f" mean={stats['mean']:.4f} std={stats['std']:.4f}")
93 print(f" Saved -> {NORM_STATS_PATH}")
94
95 print(f"\n Files saved to : {OUTPUT_DIR}")
96 print(" Data setup complete.\n")
97
98
99 if __name__ == "__main__":
100     main()

```